

Situación Didáctica

El equipo de diseño de TecLag Software han logrado diseñar las acciones semánticas que definen el comprobador de tipos estático del compilador de lenguaje PaytonTec. Ahora los diseñadores entregarán el Esquema de Traducción al equipo de desarrollo para implementar en código Java dichas acciones semánticas.

Usando el Esquema de Traducción diseñado en el paso anterior implementar el Comprobador de Tipos de lenguaje PaytonTec dentro de la clase **SintacticoSemantico**.

Cada acción semántica debe ir enmarcada antes y después con una línea de comentario que indique “Accion semántica” y “Fin acción semantica” respectivamente.

Cada acción semántica debe ir condicionada a que la variable **analizarSemantica** sea true de la siguiente manera:

```
if ( analizarSemantica ) {  
    // Accion semantica  
    .....  
    // Fin accion semantica  
}
```

EL DESARROLLO DE LA PRACTICA DEBE INCLUIR:

En la sección de Análisis mencionar:

1. Dar el algoritmo de 4 pasos visto en clase (sin incluir ejemplos).

En la sección de Diseño:

1. Dar el diseño UML de las clases SintacticoSemantico y Atributos. El diagrama de la clase SintacticoSemantico debe reflejar el argumento que se pasa en cada procedure así como mostrar las constantes VACIO y ERROR_TIPO.

En la sección Código:

1. Pegar el código de **Atributos.java** y **SintacticoSemantico.java** completas. El prologo debe llevar en la sección de “Última Modificación” el registro de las modificaciones realizadas.

En la sección de Prueba de Ejecución:

1. Usando los programas de prueba proporcionados documentar un caso de prueba por cada programa con el siguiente formato.

CASO DE PRUEBA: #1

Validacion de sentencias de asignacion

Programa de prueba

```
declare @nombre_completo char (30)  
declare @promedio1 float  
declare @calif1 int  
declare @calif2 int  
  
assign @nombre_completo := 'Fernando'  
assign @promedio1 := 0  
assign @promedio1 := 3.14159  
assign @calif1 := 10  
assign @calif2 := @calif1  
assign @promedio1 := ( @calif1 + @calif2 ) * 20  
end
```

Resultado esperado:

Programa valido, 0 errores encontradas

Resultado obtenido:

Programa no valido, 10 errores encontrados

Estatus: **OK** o **X**